

Object-Oriented Design

*Object-Oriented Software Development Implementation
for a Multi-Game Board Game Framework*

Prepared For:

Queensland University of Technology
IFQ584 - Object-Oriented Design and Development

Assignment 3: Object-oriented software development project

Due Date: Friday 25 September 2026

Prepared By:

Team 01B

Amin Noor – n12963399 – amin.edalatnoor@connect.qut.edu.au

Xander Salathe – n1250331 – xander.salathe@connect.qut.edu.au

Table of Contents

Table of Contents	1
Declaration of Contributions	2
Xander Salathe	2
Amin Noor.....	2
Other members	2
Statement of Completion	2
Requirements Completed:	2
Requirements Incomplete:.....	2
Executive Summary	3
Design Documentation.....	4
Design Overview and Summary.....	5
Overall Design	5
Changes and Improvements from Assignment 2	5
CRC Diagram	6
Full size diagrams:.....	7
Design Patterns and Principals.....	7
Design Patterns.....	8
Simple Factory: Game Creation	8
Strategy: Computer Move Selection.....	8
Template Method: Committing a Move	8
Design Principals.....	8
Controller and Separation of Concerns.....	8
Polymorphism.....	8
Single Responsibility and Encapsulation.....	9
Implementation Details	9
Program Execution	10
Prerequisites.....	10
Build and Run Instructions	10
Using the Application	10
Use of Libraries and Frameworks	10

Declaration of Contributions

Xander Salathe

My coding contributions were the implementation of the following: Enums; Piece (abstract), NumberPiece, MarkPiece; Player (abstract), HumanPlayer, ComputerPlayer; GomokuGame, NumericalTTGame, NotaktoGame; GameFactory; and MoveStrategy (interface), WinningThenRandomStrategy. Beyond this, I did substantial work on the report, including updating all diagrams with changes since A2 and codifying in Mermaid.js, generating the report layout, and majority of responses, analysis, and copy. Logistically, I set up the initial GitHub repository, and team Slack for asynchronous communications.

Amin Noor

My coding contributions were the implementation of the following:

Classes of Game (abstract), Board, MoveHistory, Move, SaveManager (saving and loading the data), GameController and Help. Throughout these I also generated the UI. Beyond that, I setup weekly and sometime more frequently cadence meetings and contributed to the current report. I further setup the second GitHub repository after having access issues with Xanders.

Other members

Other team members allocated (Yu-Duan Cheng; Happy Singer) to this group were uncontactable throughout the assignment process.

This was originally flagged with the OLA on 14/08/2026, with confirmation to proceed with the contactable group members on 25/08/2026.

Statement of Completion

Requirements Completed:

The current code implements an extensible framework with implementation for the requested 3 games while allowing new games to be easily added (via their own class) and generated using the Game Factory class.

The undo and redo functionality allows smooth transition, and move after and undo, correctly clears the moves in the redo list. Help function is always one click away and the help is contextually aware, so it shows the relevant information to the current game.

Requirements Incomplete:

All the requirements are completed.

Executive Summary

This project delivers a single extensible C# application for three two-player board games: Numerical Tic-Tac-Toe, Notakto, and Gomoku. Players can start a new game or restore a saved game; choose human-vs-human, or human-vs-computer play; and have text-based guidance throughout the process. The application validates moves, displays the current board and result, and provides contextual in-game help via commands.

Each game retains its own rules within the shared framework. Numerical Tic-Tac-Toe uses odd and even number pieces on a user configurable sized square board, with a target sum for winning lines. Notakto uses three 3x3 boards on which both players place the same mark, where the player who completes the final active board loses. Gomoku uses a 15x15 board, with a win condition of five matching marks in a row. In human-vs-computer mode, the computer selects an immediate winning move where one is available and otherwise selects a random valid move. In human-vs-human mode, two humans share the same terminal and alternate turns as prompted by the program.

The application records the full sequence of moves, so players can undo and redo individual turns. Making a new move after an undo discards the remaining move sequence. Saving preserves the game type, vs mode, boards, players, pieces, current turn, result, and move history. This allows a loaded game to continue from the same state it was left in, with move undo available. Invalid input is explained and prompted for again without changing game state.

The design separates shared gameplay from game-specific rules. The abstract Game class manages turn state, move validation, results, and history. Each concrete game class implements its own move and win rules. Board and typed Piece classes provide a common model for cells and playable pieces. GameFactory assembles the selected game and players. A move strategy interface isolates computer turn logic.

These responsibilities make the three games consistent to use and provide clear bases for extending the program through the addition of further games.

Design Documentation

Design Overview and Summary

Overall Design

The application is designed as an extensible console framework for board games - Numerical Tic-Tac-Toe, Notakto, and Gomoku. An abstract `Game` class holds shared state and behaviour. Each game then supplies its own valid moves, win conditions, result evaluation, and help text. `Board` manages cell access and placement, while `Move` records a players piece and chosen position. This structure allows the three games to share a turn model without combining their distinct rulesets into a single class.

The surrounding classes separate application functions from game rules. `GameController` holds the play loop and commands. `GameFactory` creates the given game and players. `SaveManager` handles persistence. The class diagram shows these relationships, with the object and sequence diagrams showcasing application state and interaction during use.

Changes and Improvements from Assignment 2

The team compared the two Assignment 2 designs against Assignment 3 requirements and selected Amin's design as the implementation base. Its abstract `Game`, game subclasses, `Board`, `Move`, `MoveHistory`, and `Player` class provided a framework for the required games and gameplay action. The team incorporated selected features from Xander's design where they made the framework easier to extend, without adding excessive layers of abstraction given the application scope.

Key changes:

1. Typed pieces and enums

`Piece`, `NumberPiece`, and `MarkPiece` replace the original integer piece. `Player` tracks ownership and reusability, while `Board` stores the piece objects used by `Move`. Enums represent game type, mode, and result instead of relying on string interpretation. The addition of `Piece` class allowed great flexibility in terms of extending the application with new games. We discussed the original idea of an internal numbering system, and each game representing the numbers based on its requirement but ultimately decided on having piece as a separate class, as it allows a cleaner setup with less extra work if new games are to be added with different piece representation.

2. Centralised construction

`GameFactory` creates the boards, players, pieces, and game type in one location. This removes game-specific setup from the controller and makes the configuration rules easier to extend. Previously The Game Controller was responsible for creating the selected game. The controller handles menus and input; `GameFactory` creates the game, boards, players, and pieces. The main goal here was to separate console interaction from object construction.

3. Separate computer strategy

`MoveStrategy` and `WinningThenRandomStrategy` move the immediate win with random fallback algorithm out of `ComputerPlayer`. This allows future strategy to be introduced without changing the player class/subclass or game rules.

4. Simpler Help Class

Original design had a `Help` class which was responsible in providing the help for all the `Game`. During this assignment we decided to move game specific `Help` to each `Game` concrete class, and the `Help` class becomes the controller for the help display. This helps each class to be more independent, and each class to be easier to maintain in case specific logic in a game gets updated, requiring an update to the `Help`.

These choices were made to keep the implementation feasible under time constraints, while leaving accessible extension points. A new game requires a `Game` subclass, `Piece`, `Board`, and `GameFactory` constructor case. The shared `Board`, `Player`, `MoveHistory`, and `GameController` can be reused.

CRC Diagram

Relevant classes that support polymorphism

Class	Responsibility	Collaborators
Game	Hold shared state; validate and commit moves; undo and redo; define common methods	GameController; MoveStrategy; Board; Player; Move; MoveHistory
NumericalTTTGame, NotaktoGame, GomokuGame	Override the four Game operations with each games specific legal moves, rules, result and help text	Game; Board; Move; Player; Pieces
Piece, NumberPiece, MarkPiece	Supply methods for using a number or mark piece	Player; GameController; Board; Move
Player, ComputerPlayer, HumanPlayer	Hold identity and available pieces; use and restore pieces; delegate computer control; distinguish computer player	Piece; GameController; MoveStrategy
MoveStrategy, WinningThenRandomStrategy	Define computer move selection	ComputerPlayer; Game
Board, Move, MoveHistory	Stored typed cells and moves; retain ordered history of game	Game; Player; Piece
GameController, Help	Handle commands, prompts, board display, and guidance without owning rules	Game, ComputerPlayer, GameFactory, SaveManager
GameFactory, SaveManager	Create configured games; save state as JSON, and restore.	Game(s); Board; Player; Piece; MoveStrategy; Game; Move

Full size diagrams:

Available end of file in the following order:

5. Class Diagram
6. Object Diagram 1 – Notakto game in-progress – one inactive board.
7. Object Diagram 2 – Gomoku game loaded – undo move available.
8. Sequence Diagram 1 – Gomoku - Computer plays winning move.
9. Sequence Diagram 2 – Gomoku – Load saved state and undo move.

Design Patterns and Principals

Design Patterns

Simple Factory: Game Creation

`GameFactory.CreateGame` creates a configured `Game`. Its relevant operations `CreateNumericalTTTGame`, `CreateNotaktoGame`, and `CreateGomokuGame`, construct the board and pieces. `CreatePlayers` selects a human or computer player and supplies a computer strategy if required. This centralises setup rules (eg, Notaktos three boards; Gomokus 15x15 fixed board) and eliminates repeating construction logic from the controller. Adding a game requires a new game class, and factory method, but does not require changes to the controller's game creation flow.

Strategy: Computer Move Selection

`ComputerPlayer.ChooseMove()` delegates to the `MoveStrategy.ChooseMove()` interface. `WinningThenRandomStrategy` requests valid move via the game type `GetValidMoves()`, tests candidates with `WouldMoveWin()` and chooses a winning move where one exists. If no winning move is available, a random move is chosen. The computer player and game rules remain reusable if a different move selection strategy is introduced in future extension, such as a less aggressive, or more sophisticated move strategy for the computer player.

Template Method: Committing a Move

`Game.MakeMove()` defines the common sequence for a turn: validate the move, use the players piece, apply the move, record it in `MoveHistory`, evaluate the effect on win state, and switch players if play continues. Each game overrides `IsValidMove()` and `EvaluateResult()` to specify its own rules. The shared sequence prevents the three games from implementing differing versions of move recording and turn changes, while specifying their unique win conditions in their own class allowing easier extensibility.

Design Principals

Controller and Separation of Concerns

`GameController` handles console commands and displays the current game. It uses `GameFactory` to create games, and `SaveManager` to save/load. In the sequence diagram it obtains a computer move through `ComputerPlayer.ChooseMove()`, applies through `Game.MakeMove()`, and calls `Game.UndoMove()` to undo a turn. `Game` and its subclasses manage moves, history, and rules, while `GameController` handles input and display.

Polymorphism

The abstract `Game` class lets the controller work with any of the three games through operations such as `GetValidMoves()`, `WouldMoveWin()`, `EvaluateResult()`, and `GetHelpText()`. `Player` provides shared piece operations (eg, `HasPiece`, `UsePiece`, `RestorePiece`), while the subclasses `HumanPlayer` and `ComputerPlayer` provide different player behaviour. Similarly, `NumberPiece` and `MarkPiece` implement `Piece.DisplayValue()` and `IsReusable()` according to their game rules. These common types reduce game specific checks in shared code and make additional games, players, or pieces easier to introduce.

Single Responsibility and Encapsulation

Following SOLID principals' responsibilities are assigned to specific classes. `Board` manages cell through `GetCell()`, `PlaceMove()`, and `RemoveMove()`. `MoveHistory` records moves and supports `UndoLastMove()` and `RedoLastMove()`. `Game` subclasses determine legal moves and results. `SaveManager` handles persistence. Additionally, the `MoveStrategy` interface applies dependency inversion.

Private board cells, player piece lists, and move-history state are access through defined operations rather than edited directly by the controller. This makes state changes easier to trace, and protects undo, redo, and piece ownership rules.

Following DRY principals' the design also avoids repeating logic. `Game.IsValidMove()` provides shared board and empty-cell checks. `Board.IsWithinBoard()` provides one boundary check for all games. `Board.HasSize()` provides validation for preset game boards. `Game.MakeMove()` supplies the common move process.

Implementation Details

Program Execution

Prerequisites

10. [.NET 10 SDK](#) installed on your system.
11. To confirm the installed .NET version:
12. Run the following command in your preferred terminal:
13. `dotnet --version`
14. Expected output:
15. `10.0.xxx`

Build and Run Instructions

16. Open a terminal in the project directory
17. Run the following command:
18. `cd MultiGameApp`
19. Clean previous build artifacts:
20. Run the following command:
21. `dotnet clean`
22. Build the application
23. Run the following command:
24. `dotnet build`
25. Run the application
26. Run the following command:
27. `dotnet run`

Using the Application

28. Initial game configuration requires the user to start a new game or load a saved game.
29. If starting a new game:
30. User must specify the game type:
31. `Numerical TicTacToe; Gomoku; Notakto.`
32. User must specify the opponent type:
33. `human vs human; human vs computer.`
34. User must specify the board size if required.
35. If loading a saved game:
36. User selects the Load menu item.
37. While in game a user may:
38. Request game information via the `help` command on their turn.
39. The help command advises the win-state of the specified game.
40. Play a move on a specified board.
41. When playing a move, depending on game type the user will either specify a Number, or play a predetermined Mark piece.
42. This repeats until the win state is achieved by either player, or the game ends in a draw.

Use of Libraries and Frameworks

Per the .csproj configuration - implicit using is enabled, and the project only uses .NETs built-in libraries.

Library/Framework	Use in code
System.Collections.Generic	List<T>; IReadOnlyList<T>; IEnumerable<T> in Players, Boards, Pieces, MoveHistory, Game
System.Linq	Enumerable.Range; Select; Where; ToList; OrderBy; SequenceEqual; OfType in NumericalTTTGame, GameController
System	Random; Array; Enum; Exceptions; Console throughout program
System.Text.Json	JsonSerializer, JsonSerializerOptions, JsonDocument, JsonElement in SaveManager
System.IO	Path, File, InvalidDataException in Program, SaveManager









